

Lecture 22

C Programming II

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Pointers and Memory

Memory

Memory: array of bytes

- Usually contains 2^{64} bytes (more details in CS 21/140)
- Contains all data of a running program

Memory

<u>Address (index)</u>	<u>Value (byte)</u>
0	0x41
1	0x61
2	0x62
3	0x63
4	0x00
...	. . .

Bytes as Values

Recall that a `char` is a one-byte integer in the range $[-128, 127]$ that corresponds to an ASCII symbol (for nonnegative values)

- We can see what each byte in the array corresponds to if treated as a `char`

<u>Address (index)</u>	<u>Value (byte)</u>	<u>Value (byte)</u>	<u>Value (byte)</u>
0	0x41	65	'A'
1	0x61	97	'a'

<u>Address (index)</u>	<u>Value (byte)</u>	<u>Value (byte)</u>	<u>Value (byte)</u>
2	0x62	98	'c'
3	0x63	99	'd'
4	0x00	0	'\0'
...	...	...	...

Pointers

Pointer: Variable holding a pointer value

Pointer value: Memory address or “index”

- `char *` is a `char` pointer type
- `char *p;` (uninitialized; garbage)
 - `p` is a variable containing an address
 - Said address contains a `char`

Pointers

- `char *p = (char *) 3;`
 - p contains 3
 - 3 is an address containing 0x63

Pointers can be seen as arrows pointing to memory boxes
(where boxes contain bytes)

Variables, Address-Of, Dereferencing

Variable: Alias for range of addresses associated with a type

- `char ch = 'd';`
 - Reserves an address range for `ch` (random)
 - Places `'d'` in its address range

Variables, Address-Of, Dereferencing

Address-of operator (&): Gets address (pointer value) of variable

- `&ch` means “What is the address of `ch`?”
- `char *p = &ch;`
 - Assigns the address of `ch` to `char` pointer `p`

Variables, Address-Of, Dereferencing

Dereferencing operator (): Accesses address “pointed to” by pointer*

- `*p` means “Bring me to the address `p` contains”
 - Analogous to `MEM[p]`
- `ch = *p; (MEM[&ch] = MEM[p])`
 - Assigns value that address `p` contains to `ch`
- `*p = 'x'; (MEM[MEM[p]] = x)`
 - Assigns `'x'` to address that `p` contains

Memory and Multibyte Values

Recall that `int` is usually 4 bytes; how many addresses would an `int` take up?

- How would variables, `&`, and `*` work with `ints` that span multiple addresses?

<u>Address (index)</u>	<u>Value (byte)</u>
0	0x0f
1	0x2e

<u>Address (index)</u>	<u>Value (byte)</u>
2	0x55
3	0xaa
4	0xfa
...	...

Analogy

Analogy: Boxes with birthday candles



Analogy

Bytes themselves are meaningless; we give meaning to bytes (more in CS 20/21)

Abstracted View of Memory

```
#include <stdint.h>

int main() {
    char ch = 'A';
    int nums[] = {10, 20, 30};
    int64_t x = -1;
    int y;
}
```

Abstracted View of Memory

<u>Variable</u>	<u>Address</u>	<u>Value</u>
char ch	100	'A' or 65
int nums[] , (nums[0])	101	10
nums[1]	105	20
nums[2]	109	30
int64_t x	113	-1
int y	121	

Address ranges of variables can be inferred from their starting address and their type

Pointers as Addresses

```
#include <stdint.h>

int main() {
    int x = 12;
    uint8_t nums[] = {10, 20, 30};

    int *p1 = &x;
    uint8_t *p2 = &nums[1];

    *p2 = *p1 + 1;
}
```

Pointer arrows are visual equivalents of memory addresses

Pointer Arithmetic and Indexing

Visualization: [link](#)

```
int main() {  
    int nums[] = {10, 20, 30};  
  
    for (int i = 0; i < 3; i++) {  
        // Visualize with `int *p = nums + i;`  
        *(nums + i) = *(nums + i) + 1;  
    }  
}
```

Pointer Arithmetic and Indexing

Adding **1** to a pointer makes it move to address of next element (adds the size of an element in bytes)

Pointer Arithmetic and Indexing

```
int main() {  
    int nums[] = {10, 20, 30};  
  
    for (int i = 0; i < 3; i++) {  
        nums[i] = nums[i] + 1;  
    }  
  
    for (int i = 0; i < 3; i++) {  
        *(nums + i) = *(nums + i) + 1;  
    }  
}
```



```
for (int *p = nums; p < nums + 3; p++) {  
    *p = *p + 1;  
}  
}
```

Pointer Arithmetic and Indexing

$p[i]$ is *syntactic sugar* for $*(p + i)$

- All loops above are functionally equivalent
- Array name decays to pointer to first index

Pointers as Function Calls

Passing Pointers as Arguments

```
// `void` means nothing returned  
void f(int *p) {  
    *p = *p + 1;  
}  
  
int main() {  
    int x = 11;  
    f(&x);  
}
```

Passing Pointers as Arguments

Pointers are copied as is when passed (just like all other arguments)

- Pass-by-value/Call-by-value semantics

Passing Pointers as Arguments

```
void f(int *p, int n) {  
    for (int i = 0; i < n; i++) {  
        p[i] *= 10;  
    }  
}  
  
int main() {  
    int nums[] = {10, 20, 30};  
    f(nums, 3);  
    // Technically passing a two-int array  
    f(nums + 1, 2);  
}
```

Passing Pointers as Arguments

Array name decays into pointer to first index (copied as is as well)

- Must pass array size as well
 - Only way for callee to know array bounds

Aside: Taking user input

`scanf(<format_specifier_string>, <ptr1>, <ptr2>, ...)`

- Input version of `printf` from `stdio.h` (but takes pointers instead)
- `%d` to read an `int` (more in reading assignment)

Aside: Taking user input

```
#include <stdio.h>

int main() {
    int a, b;
    printf("Enter a number: ");
    scanf("%d", &a);
    printf("Enter another number: ");
    scanf("%d", &b);

    printf("%d + %d = %d\n", a, b, a + b);
}
```

Aside: Taking user input

`scanf` returns number of items read (ignored for convenience; also useful for "%d %d", etc.)

Aside: Taking user input

```
#include <stdio.h>

int main() {
    int n;

    printf("How many numbers? ");
    scanf("%d", &n);

    int nums[n]; // Variable-length array;
                // size must be positive
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &nums[i]);  
    }  
  
    for (int i = 0; i < n; i++) {  
        printf("nums[%d]: %d\n", i, nums[i]);  
    }  
}
```

Warning: Never return pointers to local variables

```
#include <stdio.h>

int *do_not_do_this() {
    int x = 2025;

    return &x;  // Allowed by C; mortal sin
}

int main() {
    int *p1 = do_not_do_this();
    printf("This should be 2025: %d\n", *p1);
}
```

```
int *p2 = do_not_do_this();  
printf("Hello, world!\n");  
printf("This should be 2025: %d\n", *p2);
```

```
int *p3;  
printf("Pointer value: %p\n", p3);  
printf("Dereferenced: %d\n", *p3);
```

```
}
```

Warning: Never return pointers to local variables

- Whenever a function is called, memory is allocated to it for local variables and other details (more in CS 21)
- When a function call is done, the allocated memory is deallocated
- Accessing deallocated memory leads to undefined behavior; never do this!

Incorrect way of returning an array

21/return_array_wrong.c

Incorrect Way of Returning an Array

Recall that all local variable data is deallocated when owning function is done

Heisenbug: Bug in which behavior changes when “observed” (or when something unrelated is added)

- Case 1: Run as is
- Case 2: Add `printf` before for loop of main

Possible way of “returning” an array

21/return_array_ok.c

Possible way of “returning” an array

- Caller supplies array via pointer argument as “output parameter” instead; no need to return anything (callee can update array directly)
- Another way is by using *dynamic memory allocation*

Null Pointer

Example

Task: Define a function `int *first_negative(int *, int)` that returns a pointer to the leftmost negative integer of the `int` array passed as a `int *` with its size as the second argument

Example

```
int *first_negative(int *arr, int n) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] < 0) {  
            return &arr[i];  
        }  
    }  
  
    return ???; // Must be consistent with return type  
}
```

Example

What should be returned if the array contains no negative integers?

- Why is using a sentinel value here a bad idea?
- Is there an equivalent for **None** or **Nothing** in C?
- What should the return type be?

Null Pointer

NULL: Macro defined as special address value (usually 0) that represents the absence of a value being pointed to (i.e., arrow pointing to “nothing”)

Null Pointer

```
#include <stdio.h>
#include <stdlib.h>

int *first_negative(int *arr, int n) {
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            return &arr[i];
        }
    }

    return NULL;
}
```

}

Null Pointer

```
int main() {  
    int nums[] = {10, 20, 30};  
    int *p = first_negative(nums, 3);  
  
    if (p == NULL) {  
        printf("No negative found\n");  
    } else {  
        printf("First negative: %d\n", *p);  
        printf("Address: %p\n", p);  
    }  
}
```

Null Pointer

- NULL can be assigned to any pointer type
- Conventional to use as initialization value

Segmentation Fault

Segmentation fault: Runtime error wherein a program attempts to access a memory address it has no permission to (CS 140)

- Shortened to segfault; memory “segment” illegally accessed
- Normally not a concern in Python and Elm (still present in special cases)

Segfault examples

```
#include <stdlib.h>

int main() {
    int *p = NULL;
    int x = *p;    // Dereferencing a null pointer
}                // is not allowed (segfault)
```

Segfault examples

```
int main() {  
    int x = 12;  
    int *p = &x + 10000000; // Likely restricted;  
                             // no segfault yet  
  
    int y = *p;             // Segfault on access  
}
```

Segfault examples

```
int main() {  
    int *p;           // Uninitialized; contains garbage  
  
    int x = *p;       // Address is garbage;  
}                     // likely restricted (segfault)
```


Segfault examples

```
int main() {                                // char str[] = "...";  
    char *str = "Hello!"; // can be modified  
  
    str[5] = '?'; // Segfault; char * pointing  
                  // to "..." are read-only  
}
```

Segfault examples

```
int main() {  
    int nums[] = {10, 20, 30};  
  
    int x = nums[100];    // May or may not be valid  
    int y = nums[10000];  // Likely invalid; segfault  
}
```

Segfault examples

```
int main() {
    // `s` is missing null terminator
    char s[] = {'H', 'e', 'l', 'l', 'o', '!'};

    printf("%s\n", s); // Reads out-of-bounds until
                        // segfault or garbage null
}
```

Multiple levels of indirection

Pointers can point to other pointers

Multiple levels of indirection

```
#include <stdio.h>
int main() {
    int x = 12;           // Int
    int *p = &x;          // Pointer to int
    int **pp = &p;        // Pointer to (pointer to int)
    int ***ppp = &pp;     // Pointer to (pointer to pointer to int)
    printf("    x = %d\n", x);      // 12
    printf("    *p = %d\n", *p);    // 12
    printf("    **pp = %d\n", **pp); // 12
    printf("    ***ppp = %d\n", ***ppp); // 12
}
```

Reading assignment: user input details

`scanf` and the input buffer

Input buffer: Stores entered user input as sequence of `chars`; consumed by `scanf`

scanf and the input buffer

%d (into `int *`)

1. Eat all leading whitespace (Space, `'\n'`, `'\t'`, etc.)
2. If next char is `+` or `-`:
 - Eat it and add to input
3. While next char is digit:
 - Eat it and add to input
4. Stop at first nondigit (not eaten)

`'\n'` from pressing Enter gets left in input buffer!

scanf and the input buffer

`%c` (into `char *`)

- Eat next `char` (any) in buffer and return as input

scanf and the input buffer

`%s` (into `char []`)

1. Eat all leading whitespace
2. While next char is not whitespace:
 - Eat it and add to input
3. Stop at whitespace (uneaten)

`'\n'` also gets left in input buffer!

scanf and the input buffer

```
#include <stdio.h>

// 100\n200\n300\n400
// 100 200 300 400\n
// 100 200 300\n400
// 100 x 200\n
// 100 xy 200\n
int main() {
    int a = 10, b = 20, c = 30;
    char ch, s[100];
```

```
scanf("%s", s);  
scanf("%d", &a);  
scanf("%d", &b);  
scanf("%c", &ch);  
scanf("%d", &c);
```

```
printf("s: [%s]\n", s);  
printf("a: %d\n", a);  
printf("b: %d\n", b);  
printf("ch: [%c]\n", ch);  
printf("c: %d\n", c);
```

```
}
```

fgets

`fgets(<char_ptr>, <n>, <input_stream>)`

- Eats characters from <input_stream>
- Stops when:
 - `'\n'` is eaten
 - `n - 1` characters are eaten
 - Input stream is closed
- Always appends `'\0'` to output

`fgets`

- Easier way to read entire line of input into string
- Easier to make array and input sizes consistent
- `stdin` is standard input stream (`stdio.h`)

fgets

- `atoi(<array>)` of `stdlib.h` returns `int` equivalent of null-terminated string
 - Also see `sscanf` here as alternative

fgets

```
#include <stdio.h>
#include <stdlib.h>

// 123\n456\n
// 12345678\n
// 123 456 789\n
// 1234567\n123
int main() {
    char buffer[8], other[100];

    fgets(buffer, 8, stdin);
    int a = atoi(buffer);
```



```
int b;  
scanf("%d", &b);
```

```
// Note that input will not pause here  
fgets(other, 100, stdin);
```

```
printf("buffer: [%s]\n", buffer);  
printf("a: %d\n", a);  
printf("b: %d\n", b);  
printf("other: [%s]\n", other);
```

```
}
```